ProjectMind

Complete Beginner-Friendly Documentation

Autonomous AI-Powered Developer Memory & Documentation Engine

10 Modules | **259 Tests** | **Python 3.12** | **Local-First AI**

Modules: Foundation · Watcher · AI · Analysis · Docs · Graph · Memory · Obsidian · Git · Intelligence

Generated 2026 · ProjectMind v0.1.0

ProjectMind — Complete Beginner-Friendly Documentation

> ***Think of this document as a classroom lecture.*** *We will go through the entire*
>
> *project from zero, explaining every concept, file, class, function, and line of*
>
> *logic as if you are reading this for the very first time.*

Table of Contents

1. Project Overview

What is ProjectMind?

Imagine you are a developer working on a large codebase — hundreds of Python files, complex function relationships, git history stretching back years. You constantly have to ask yourself:

- *"What does this file do?"*
- *"Which files depend on this function?"*
- *"What changed in last week's commits?"*
- *"What code smells should I fix?"*

ProjectMind is an autonomous AI system that answers all of these questions automatically — without you asking. It watches your codebase in real time, analyzes every file, builds a knowledge graph of your project, writes documentation, stores everything as searchable memory, monitors git history, and even proactively suggests refactors.

One-sentence summary

> *ProjectMind is a **local-first AI brain** that watches your code and builds a living,*
>
> *searchable, documented knowledge graph of your project — automatically.*

Purpose

Pain Point	How ProjectMind Solves It
Documentation is always out of date	Auto-generates docs on every file change
Hard to understand how files connect	Builds a live dependency graph
Forgot why a commit was made	AI summarises every git commit
Code smells pile up unnoticed	Detects anti-patterns on a schedule
Can't search your own codebase by meaning	Semantic memory search via ChromaDB
Notes scattered across tools	Writes structured Obsidian-compatible markdown

---

Features

- **File Watcher** — monitors your source directories for any file change
- **AST Code Analysis** — reads Python files and extracts functions, classes, imports, complexity
- **AI Integration** — connects to a locally-running Ollama/Qwen model for summaries and suggestions
- **Auto-Documentation** — generates rich markdown docs with changelogs for every file
- **Dependency Graph** — builds a networkx directed graph showing which files import which
- **Semantic Memory** — chunks and embeds all analysis results into ChromaDB for search
- **Obsidian Vault** — writes everything as `[[wikilink]]`-connected markdown notes
- **Git Monitor** — watches for new commits and stores AI-written summaries
- **Intelligence Engine** — runs every hour to detect patterns, cycles, hotspots, and generate refactor suggestions
- **Codebase Q&A** — answer natural-language questions about your codebase using memory retrieval

---

2. Folder and File Structure

```
ProjectMind/
■
■■■ main.py                 ← Entry point — starts the whole application
■■■ pyproject.toml          ← Project metadata + pytest configuration
■■■ requirements.txt        ← Runtime dependencies (pip install -r requirements.txt)
■■■ requirements-dev.txt    ← Dev dependencies (pytest, etc.)
■
■■■ config/                 ← Configuration files (YAML)
■   ■■■ default_config.yaml ← Factory defaults for every setting
■   ■■■ config.yaml         ← Your personal overrides (git-ignored)
■   ■■■ config.example.yaml ← Copy this to config.yaml to get started
■
■■■ core/                   ← M1: Foundation — infrastructure all other modules use
■   ■■■ __init__.py         ← Public exports (EventBus, Settings, get_logger, etc.)
■   ■■■ config.py           ← Config loader: reads YAML + env vars → Settings dataclass
■   ■■■ logger.py           ← Logging setup: rotating file + colored console
■   ■■■ registry.py         ← Service registry: a dictionary of all live services
■   ■■■ bootstrap.py        ← Wires everything together at startup
■   ■■■ event_bus.py        ← Pub/sub bus for module communication
■   ■■■ interfaces.py       ← Abstract base classes (contracts) for every service
■   ■■■ exceptions.py       ← Custom exception classes
■   ■■■ utils.py            ← Small helpers (ensure_dir, etc.)
■
■■■ watcher/                ← M2: Filesystem monitoring
■   ■■■ __init__.py
■   ■■■ events.py           ← FileChangeEvent dataclass + ChangeKind enum
■   ■■■ filters.py          ← Decides which files/dirs to watch or ignore
■   ■■■ file_tracker.py     ← Debounce: waits for a file to "settle" before acting
■   ■■■ watcher.py          ← watchdog event handler that feeds the tracker
■   ■■■ watcher_manager.py  ← The WatcherManager service (start/stop)
■
■■■ ai/                     ← M3: AI communication (Ollama/Qwen)
■   ■■■ __init__.py
■   ■■■ ai_manager.py       ← AIManager class — sends prompts, handles fallback
■   ■■■ prompt_registry.py  ← Stores named prompt templates
■   ■■■ response_parser.py  ← Extracts JSON from AI responses
■
■■■ analysis/               ← M4: Static code analysis
■   ■■■ __init__.py
■   ■■■ analysis_types.py   ← FunctionInfo + FileAnalysis dataclasses
```

```
■   ■■■ analyzer_engine.py   ← Service: subscribes to watcher events, runs analysis
■   ■■■ ast_analyzer.py      ← Reads Python source → AST → extracts structure
■   ■■■ complexity.py        ← Calculates cyclomatic complexity
■   ■■■ dependency_mapper.py ← Finds which local modules each file imports
■
■■■ docs/                    ← M5: Documentation generation
■   ■■■ __init__.py
■   ■■■ frontmatter.py       ← Builds the YAML --- block at the top of a note
■   ■■■ doc_generator.py     ← Assembles full markdown from FileAnalysis
■   ■■■ changelog.py         ← Diffs two FileAnalysis objects → changelog entries
■   ■■■ template_engine.py   ← Jinja2 template rendering
■   ■■■ doc_engine.py        ← Service: subscribes to analysis events, publishes docs
■
■■■ graph/                   ← M6: Dependency graph
■   ■■■ __init__.py
■   ■■■ graph_builder.py     ← Wraps networkx.DiGraph with add/remove/query methods
■   ■■■ graph_state.py       ← Saves/loads graph to/from disk (JSON)
■   ■■■ graph_analyzer.py    ← find_orphans, find_hubs, find_cycles, hotspots
■   ■■■ graph_engine.py      ← Service: subscribes to analysis events, updates graph
■
■■■ memory/                  ← M7: Semantic memory (ChromaDB)
■   ■■■ __init__.py
■   ■■■ chunker.py           ← Splits FileAnalysis into overlapping text chunks
■   ■■■ embedder.py          ← sentence-transformers → float vectors
■   ■■■ memory_store.py      ← ChromaDB wrapper: upsert, delete, search
■   ■■■ memory_updater.py    ← Service: subscribes to analysis events, upserts chunks
■   ■■■ semantic_search.py   ← Query interface: search(query, top_k)
■
■■■ obsidian/                ← M8: Obsidian vault writer
■   ■■■ __init__.py
■   ■■■ vault.py             ← VaultManager: write_note / read_note / list_notes
■   ■■■ vault_index.py       ← In-memory index of all notes (stem → path)
■   ■■■ vault_writer.py      ← Async write queue with atomic file operations
■   ■■■ link_resolver.py     ← Converts file paths → [[wikilinks]]
■   ■■■ note_builder.py      ← Assembles the final note: doc + graph links + neighbours
■   ■■■ markdown.py          ← Markdown formatting helpers
■   ■■■ obsidian_engine.py   ← Service: subscribes to doc/graph events, writes notes
■
■■■ git_integration/         ← M9: Git history monitoring
■   ■■■ __init__.py
■   ■■■ git_types.py         ← CommitInfo + CommitSummary dataclasses
■   ■■■ git_monitor.py       ← Polls `git log`, publishes git.commit events
■   ■■■ commit_summarizer.py ← Sends diff to AI → structured CommitSummary
■   ■■■ git_memory.py        ← Stores commit summaries in ChromaDB
■   ■■■ git_engine.py        ← Service: wires monitor + summarizer + memory
■
■■■ intelligence/            ← M10: Autonomous pattern analysis
■   ■■■ __init__.py
■   ■■■ intelligence_types.py← Pattern + Suggestion dataclasses
■   ■■■ pattern_detector.py  ← Finds anti-patterns in the graph + analysis data
■   ■■■ refactor_suggester.py← Asks AI to generate refactor suggestions per pattern
■   ■■■ suggestion_store.py  ← Persists + deduplicates suggestions to disk
■   ■■■ intelligence_engine.py← Service: runs periodic analysis cycle, Q&A
■
■■■ templates/               ← Markdown templates for vault notes
■■■ vault/                   ← The generated Obsidian vault (git-ignored)
■■■ logs/                    ← Rotating log files (git-ignored)
■■■ tests/                   ← pytest test suite (259 tests)
    ■■■ test_config.py
    ■■■ test_ai.py
    ■■■ test_analysis.py
    ■■■ ...etc
```

3. Technology Stack

Language: Python 3.12

Python is used because:

- It has the best ecosystem for AI/ML (sentence-transformers, chromadb, ollama)
- `ast` module lets us parse Python source code natively — no extra tools needed
- `watchdog` library works cross-platform for filesystem events
- `dataclasses` make clean data structures
- `threading` handles background tasks simply

Key Libraries

Library	Purpose	Why This One?
`ollama`	Talks to the local Ollama server to run AI models	Official Python client for Ollama
`PyYAML`	Reads `.yaml` config files	Simplest YAML library for Python
`watchdog`	Detects file system changes (create/modify/delete)	Cross-platform, battle-tested
`Jinja2`	Renders text templates (fills `{{ variable }}` slots)	The standard Python templating engine
`networkx`	Builds and analyzes directed graphs	The de facto graph library in Python
`chromadb`	Stores and searches vector embeddings (semantic memory)	Easiest embedded vector database
`sentence-transformers`	Converts text into numerical vectors (embeddings)	Best open-source embedding models
`gitpython / subprocess`	Reads git history	Reads raw git log output
`pytest`	Runs automated tests	The standard Python test framework

External Service: Ollama

Ollama is a local server that runs AI language models (like Qwen, Llama, Mistral) on your own machine. ProjectMind sends it prompts and gets text responses back. No cloud, no API keys — everything stays on your machine.

```
Your Code → ProjectMind → Ollama Server (localhost:11434) → Qwen Model → Response
```

4. Core Concepts

Concept 1: Event-Driven Architecture

Imagine a postal system. Instead of one person calling another directly, everyone sends and receives letters through a central post office (the EventBus).

```
Module A (sender)          EventBus             Module B (listener)
     ■                        ■                        ■
     ■■■ publish("news") ■■■  ■■■ dispatch to list ■■■ ■
     ■                        ■                        ■ handler called
```

Why this matters: Module A doesn't need to know Module B exists. They are completely decoupled. You can add, remove, or replace B without touching A.

In ProjectMind:

- The **Watcher** publishes `watcher.file_change` when a file changes
- The **Analyzer** subscribes to this and publishes `analysis.file_analyzed`
- The **Doc Engine** subscribes to `analysis.file_analyzed` and publishes `docs.doc_updated`
- The **Obsidian Engine** subscribes to `docs.doc_updated` and writes the note

Nobody talks to anybody directly. Everything flows through the EventBus.

Concept 2: Dependency Injection via Service Registry

Think of the ServiceRegistry like a hotel concierge desk. You register your things at check-in, and anyone can ask the concierge for them later.

```
# At startup (bootstrap.py):
registry.register(AIClient, ai_manager)        # "Store the AI manager under AIClient key"
registry.register("vault", vault_manager)      # "Store vault under the string key 'vault'"

# Anywhere in the codebase:
ai = registry.get(AIClient)                    # "Give me the AIClient"
vault = registry.get("vault")                  # "Give me the vault"
```

Why not just import? If `analysis.py` imports `ai_manager.py` directly, and `ai_manager.py` imports from `core.py`, and `core.py` tries to import from `analysis.py`, you get a **circular import** crash. The registry breaks this cycle.

Concept 3: Abstract Interfaces (Contracts)

In `core/interfaces.py`, we define abstract classes like `AIClient`, `FileWatcher`, `MemoryEngine`. These are **contracts** — they say "whatever implements me MUST have these methods."

```
class AIClient(Service):
    def complete(self, prompt_name: str, variables: dict) -> str: ...
    def complete_raw(self, prompt_text: str) -> str: ...
    def is_available(self) -> bool: ...
```

The real `AIManager` in `ai/ai_manager.py` implements all of these. Tests can use a fake that also implements them — swapped in via `registry.register(AIClient, FakeAI)`.

Concept 4: Cyclomatic Complexity

This is a number that measures how "complex" a function is by counting its decision points (if/else, for, while, try/except, and/or).

Example:

```
def simple(x):        # complexity = 1 (no branches)
    return x + 1

def complex_fn(x, y): # complexity = 3
    if x > 0:         # +1
        if y > 0:     # +1
            return x + y
    return 0
```

Higher complexity = harder to understand, test, and maintain. ProjectMind flags functions with complexity > threshold as potential hotspots.

Concept 5: AST (Abstract Syntax Tree)

When Python reads your code, it builds a tree structure representing every piece of syntax. ProjectMind uses Python's built-in `ast` module to parse source code without running it.

```
Source:  def foo(x): return x + 1

AST:     FunctionDef
```

```
            name = "foo"
            args = [arg(arg="x")]
            body = [Return(value=BinOp(...))]
```

ProjectMind walks this tree to find: function names, parameter names, which functions are called inside other functions, whether docstrings exist, and cyclomatic complexity.

Concept 6: Vector Embeddings (Semantic Memory)

Text embeddings convert sentences into lists of numbers (vectors). Similar sentences get similar vectors. This allows **semantic search** — searching by *meaning*, not just exact words.

```
"Parse JSON from response"      → [0.12, -0.45, 0.78, ...]
"Extract data from API reply"   → [0.14, -0.42, 0.75, ...]   ← close in vector space!
"Cook pasta for dinner"         → [0.91, 0.23, -0.33, ...]   ← far away
```

ChromaDB stores these vectors and finds the closest ones to a query.

Concept 7: Wikilinks ([[double brackets]])

Obsidian uses `[[filename]]` syntax to create clickable links between notes. When ProjectMind writes a note about `parser.py`, it automatically adds links like `[[utils]]` if `parser.py` imports `utils.py`. This makes the vault navigable like a wiki.

Concept 8: Debouncing

When you save a file while typing, your editor may trigger dozens of save events in one second. Debouncing means: "wait until the file hasn't changed for X seconds, THEN process it." This prevents flooding the analysis pipeline with noise.

```
Raw events:  MODIFIED  MODIFIED  MODIFIED  MODIFIED  [2 sec pause]  → ONE event sent
```

---

5. Code Explanation — Module by Module

M1 — Foundation Engine

`core/config.py` — Configuration System

Purpose: Load settings from YAML files and environment variables and expose them as typed Python dataclasses.

How it works (3-layer merge):

```
Layer 1: config/default_config.yaml   (always loaded — provides all defaults)
         +
Layer 2: config/config.yaml          (user overrides — git-ignored)
         +
Layer 3: PROJECTMIND_SECTION__KEY=val (environment variable overrides — highest priority)
         =
Layer 4: Settings dataclass          (the final merged result used by all code)
```

Key classes:

```
@dataclass
class Settings:
    app: AppSettings          # name, version, instance_id
    paths: PathSettings       # project_root, logs_dir, vault_dir
    logging: LoggingSettings  # level, max_bytes, backup_count, console_color
    vault: VaultSettings      # sections list, frontmatter defaults
    ai: AISettings            # ollama_host, default_model, fallback_model, timeout
    watcher: WatcherSettings  # enabled, watch_dirs, debounce_seconds
    analysis: AnalysisSettings# enabled, batch_size, max_file_size
    docs: DocsSettings        # enabled, max_changelog_entries
    graph: GraphSettings      # enabled, format
    memory: MemorySettings    # enabled, chroma_db_path, embedding_model
    obsidian: ObsidianSettings# enabled
    git: GitSettings          # enabled, repo_path, poll_interval_seconds
    intelligence: IntelligenceSettings # enabled, cycle_interval_seconds, min_severity
```

`ConfigLoader.load()` **step by step:**

- `_load_defaults()` — reads `default_config.yaml` using PyYAML into a raw dict
- `_merge_user_config(raw)` — deep-merges user's `config.yaml` on top (user wins)
- `_apply_env_overrides(raw)` — scans all `PROJECTMIND_*` environment variables, converts double-underscore to nesting (`PROJECTMIND_AI__TIMEOUT=60` sets `ai.timeout = 60`)
- `_validate(raw)` — checks required fields (e.g., logging level must be valid)
- `_build_settings(raw)` — converts the raw dict into nested dataclass instances

Environment variable example:

```
PROJECTMIND_LOGGING__LEVEL=DEBUG     → settings.logging.level = "DEBUG"
PROJECTMIND_WATCHER__ENABLED=true    → settings.watcher.enabled = True
PROJECTMIND_AI__TIMEOUT=60           → settings.ai.timeout = 60
```

`core/event_bus.py` — Pub/Sub Bus

Purpose: Allow modules to communicate without importing each other.

```
class EventBus:
    def __init__(self):
        self._handlers = defaultdict(list)  # event_name → [list of handler functions]
        self._lock = RLock()                # thread-safe (reentrant lock)

    def subscribe(self, event_name, handler):
        # Add handler to the list for this event
        self._handlers[event_name].append(handler)

    def unsubscribe(self, event_name, handler):
        # Remove handler from the list
        self._handlers[event_name].remove(handler)

    def publish(self, event_name, payload=None):
        # Get snapshot of handlers (thread-safe copy)
        handlers = list(self._handlers.get(event_name, []))
        # Call each handler with the payload dict
        for handler in handlers:
            handler(payload or {})
```

Why `RLock` (Reentrant Lock)? Multiple threads may subscribe/publish concurrently. The lock ensures only one thread modifies the handler list at a time. Reentrant means the same thread can acquire it again (needed if a handler itself calls publish).

Dry run — file change event:

```
1. WatcherManager detects "src/parser.py" was modified
2. bus.publish("watcher.file_change", {"event": FileChangeEvent(path="src/parser.py", ...)})
3. EventBus looks up handlers for "watcher.file_change": [analyzer_handler, ai_handler]
4. analyzer_handler({"event": ...}) is called → queues file for analysis
5. ai_handler({"event": ...}) is called → logs the event
```

`core/registry.py` — Service Registry

Purpose: A thread-safe dictionary that maps keys (types or strings) to service instances. Acts as the dependency injection container for the whole application.

```
class ServiceRegistry:
    def __init__(self):
```

```
        self._services = {}       # key → instance
        self._lock = RLock()      # thread-safe

    def register(self, key, instance, *, replace=False):
        # key can be a type (like AIClient) or a string (like "vault")
        # If key already exists and replace=False → raises RegistryError
        self._services[key] = instance

    def get(self, key):
        # Returns the stored instance or raises RegistryError with helpful message
        return self._services[key]  # raises if missing
```

Example — why type keys are better than string keys:

```
# String key — typo-prone:
registry.register("ai_manager", ai)
ai = registry.get("ai_manger")  # typo! KeyError at runtime

# Type key — safe:
registry.register(AIClient, ai)
ai = registry.get(AIClient)     # IDE can autocomplete, typos caught early
```

`core/exceptions.py` — Exception Hierarchy

Purpose: Custom exceptions make it clear WHERE an error originated.

```
Exception (Python built-in)
■■■ ProjectMindError          ← catch ALL ProjectMind errors with one except
    ■■■ ConfigError           ← bad YAML, missing required field
    ■■■ RegistryError         ← service not found, duplicate registration
    ■■■ VaultError            ← can't write/read a note file
    ■■■ BootstrapError        ← startup failed (wraps unexpected errors)
    ■■■ WatcherError          ← watchdog can't start
    ■■■ AIError               ← Ollama unreachable, model not found, timeout
    ■   ■■■ PromptNotFoundError← asked for a prompt template that doesn't exist
    ■   ■■■ ResponseParseError ← AI returned something we couldn't parse
    ■■■ MemoryError           ← ChromaDB / embedder failure
```

`core/logger.py` — Logging System

Purpose: Set up Python's `logging` module with two outputs:

- **Console** — colored output with INFO+ messages
- **Rotating file** — all messages including DEBUG, rotates when file gets big

```
def bootstrap(logs_dir, settings):
    # Create the file handler (writes to logs/projectmind.log)
    file_handler = RotatingFileHandler(
        filename=logs_dir / settings.filename,
        maxBytes=settings.max_bytes,    # e.g., 5MB
        backupCount=settings.backup_count  # keeps 5 old files
    )
```

```
    file_handler.setLevel(logging.DEBUG)  # capture everything

    # Create the console handler (prints to terminal)
    console_handler = StreamHandler()
    console_handler.setLevel(level)       # e.g., INFO

    # Add colored formatter if console_color=True
    if settings.console_color:
        console_handler.setFormatter(ColoredFormatter(...))
```

`get_logger(__name__)` — this is used at the top of every module:

```
log = get_logger(__name__)   # returns logging.getLogger("projectmind.ai.ai_manager")
log.info("Started!")         # → "[2026-06-17 21:00:00] INFO  projectmind.ai.ai_manager: Started!"
```

All module loggers live under the `projectmind` namespace, so one call to `logging.getLogger("projectmind").setLevel(DEBUG)` changes all of them at once.

`core/bootstrap.py` — Application Startup Orchestrator

Purpose: The single place that knows about every module and wires them together.

Step-by-step bootstrap sequence:

```
Step 1: Load Settings (ConfigLoader)
        ↓
Step 2: Resolve project_root path
        ↓
Step 3: Set up logging (create logs dir, configure handlers)
        ↓
Step 4: Create Vault (create vault dir + subdirectories)
        ↓
Step 5: Create ServiceRegistry
        ↓
Step 6: Register core services:
        settings, registry, vault, project_root, logs_dir, event_bus
        ↓
Step 7: Create AIManager → register as AIClient
        ↓
Step 8: If watcher.enabled: create WatcherManager → register as FileWatcher
        ↓
Step 9: If analysis.enabled: create Module4AnalyzerEngine → register as Analyzer
        ↓
Step 10: If docs.enabled: create Module5DocEngine → register as DocumentationGenerator
         ↓
Step 11: If graph.enabled: create Module6GraphEngine → register as GraphBuilder
         ↓
Step 12: If memory.enabled: create MemoryStore + MemoryUpdater → register as MemoryEngine
         ↓
Step 13: If obsidian.enabled: create ObsidianEngine → register as "obsidian"
         ↓
Step 14: If git.enabled: create GitEngine → register as "git"
         ↓
Step 15: If intelligence.enabled: create IntelligenceEngine → register as "intelligence"
         ↓
Step 16: Register shutdown hooks (LIFO — last registered, first called)
         ↓
```

```
Step 17: Install SIGINT/SIGTERM handlers (Ctrl+C → graceful shutdown)
         ↓
Step 18: Return Application object to main()
```

The `Application` dataclass holds:

- `settings` — the loaded config
- `registry` — all registered services
- `project_root` — the absolute path to the project
- `_shutdown_hooks` — list of callables to run on exit

Graceful shutdown (LIFO order):

```
def shutdown(self):
    while self._shutdown_hooks:
        hook = self._shutdown_hooks.pop()  # pop = last in, first out
        try:
            hook()   # call stop() on each service
        except Exception:
            log.exception("Shutdown hook failed")  # log but continue!
```

The "log but continue" approach ensures one failed service doesn't prevent others from shutting down cleanly.

M2 — Watcher Engine

`watcher/events.py` — Data Structures

```
class ChangeKind(Enum):
    CREATED  = "created"
    MODIFIED = "modified"
    DELETED  = "deleted"
    MOVED    = "moved"

@dataclass(frozen=True)
class FileChangeEvent:
    path: Path        # absolute path to the changed file
    kind: ChangeKind  # what happened
    old_path: Path | None  # only set for MOVED events
    timestamp: float  # when it happened (unix timestamp)
```

`watcher/filters.py` — File Filtering

Purpose: Decide which file system events are relevant.

Two main checks:

- **Extension filter** — only pass events for `.py`, `.js`, `.ts`, `.md`, `.json`, etc.

- **Path filter** — ignore events from `node_modules/`, `.git/`, `__pycache__/`, `dist/`, `build/`, `.venv/`, `logs/`, `vault/` etc.

```
def should_process(path: Path, kind: ChangeKind, settings: WatcherSettings) -> bool:
    # DELETED events pass without extension check (file is gone, can't read it)
    if kind == ChangeKind.DELETED:
        return not _is_ignored_path(path, settings)

    # Must have a watched extension
    if path.suffix not in settings.watch_extensions:
        return False

    # Must not be in an ignored directory
    return not _is_ignored_path(path, settings)
```

`watcher/file_tracker.py` — Debounce Logic

Purpose: Prevent processing the same file multiple times in quick succession.

How it works:

```
class FileTracker:
    def __init__(self, debounce_seconds: float):
        self._pending: dict[Path, FileChangeEvent] = {}  # path → latest event
        self._timestamps: dict[Path, float] = {}         # path → last-seen time

    def see(self, event: FileChangeEvent) -> None:
        # Record the event, overwriting any previous pending event for this path
        self._pending[event.path] = event
        self._timestamps[event.path] = event.timestamp

    def flush(self, now: float) -> list[FileChangeEvent]:
        # Return events that have been "quiet" for debounce_seconds
        ready = []
        for path, event in list(self._pending.items()):
            age = now - self._timestamps[path]
            if age >= self._debounce_seconds:
                ready.append(event)
                del self._pending[path]
        return ready
```

Merge logic: If a file is MODIFIED then immediately DELETED, we prefer DELETED.

```
def _merge(existing: FileChangeEvent, new: FileChangeEvent) -> FileChangeEvent:
    if new.kind == ChangeKind.DELETED:
        return new   # DELETED wins over everything
    if existing.kind == ChangeKind.CREATED and new.kind == ChangeKind.MODIFIED:
        return existing  # CREATED + MODIFIED = still just CREATED
    return new
```

`watcher/watcher_manager.py` — WatcherManager Service

Purpose: The main service class for M2. Starts watchdog observers for each

watch directory, runs a background thread that flushes debounced events and publishes them to the EventBus.

```
class WatcherManager(FileWatcher):
    def start(self):
        # For each watch_dir in settings:
        for dir_name in self.settings.watch_dirs:
            path = self.project_root / dir_name
            if path.exists():
                observer = Observer()
                observer.schedule(handler, str(path), recursive=True)
                observer.start()

        # Start background thread that flushes debounced events
        self._flush_thread = Thread(target=self._flush_loop)
        self._flush_thread.start()

    def _flush_loop(self):
        while not self._stop.is_set():
            now = time.time()
            events = self._tracker.flush(now)
            for event in events:
                self._bus.publish("watcher.file_change", {"event": event})
            time.sleep(0.1)  # check 10 times per second
```

M3 — AI Communication Engine

`ai/prompt_registry.py` — Prompt Templates

Purpose: Store named prompt templates that can be filled with variables.

A **prompt template** has two parts:

- **System prompt** — instructions to the AI about its role ("You are a code reviewer...")
- **User prompt** — the actual question with variables filled in ("Analyze this code: ...")

```
class PromptTemplate:
    name: str
    version: str
    system: str   # "You are an expert Python code reviewer..."
    template: str # "Analyze the following code:\n\nFile: {{ file_path }}\n\n{{ code }}"

class PromptRegistry:
    def register(self, template: PromptTemplate) -> None:
        # Store template by name
        self._templates[template.name] = template

    def render(self, name: str, variables: dict) -> tuple[str, str]:
        # Fill {{ variable }} slots using Jinja2
        template = self._templates[name]
        env = jinja2.Environment()
        user_prompt = env.from_string(template.template).render(**variables)
        return template.system, user_prompt
```

Built-in prompts registered at startup:

Prompt Name	Purpose
`code_analysis`	Analyze a Python file → return JSON with `purpose` and `suggestions`
`doc_generation`	Generate extended description for a file
`commit_summary`	Summarize a git commit diff → JSON with `summary`, `impact`, etc.
`refactor_suggestion`	Given a pattern, suggest concrete refactor steps

`ai/ai_manager.py` — AIManager

Purpose: The one-and-only gateway to the Ollama AI server. All modules call `get_ai().complete("prompt_name", variables)` — they never touch Ollama directly.

Key variables:

```
self._host            # "http://localhost:11434"
self._model           # "qwen2.5-coder:14b" (primary model)
self._fallback_model  # "qwen2.5-coder:14b" (used if primary fails)
self.active_model     # which model was actually used (may differ after fallback)
self._client          # ollama.Client (synchronous)
self._async_client    # ollama.AsyncClient (for async calls)
self._registry        # PromptRegistry (holds all prompt templates)
```

`complete()` step by step:

```
1. Render the prompt: registry.render("code_analysis", {"file_path": ..., "code": ...})
   → returns (system_prompt, user_prompt) strings

2. Call _chat_with_fallback(system_prompt, user_prompt, model="qwen2.5-coder:14b")

3. _chat() sends to Ollama:
   client.chat(model="qwen2.5-coder:14b", messages=[
       {"role": "system", "content": system_prompt},
       {"role": "user",   "content": user_prompt},
   ])

4. If Ollama says "model not found" → _should_fallback() returns True
   → retry with fallback_model

5. _response_text(response) extracts the text from response.message.content

6. Log the call (model, latency in ms, token count)

7. Return the text string
```

Fallback logic:

```
def _should_fallback(self, exc: AIError, model: str) -> bool:
    # Only fallback if:
    # 1. We haven't already fallen back (model != fallback_model)
```

```
    # 2. The error looks like "model not found"
    return model != self._fallback_model and _is_model_missing(exc)
```

`ai/response_parser.py` — JSON Extraction

Purpose: AI models sometimes wrap JSON in markdown code fences or add extra text.

This module strips all that and returns just the JSON object.

```
def parse_json_object(text: str) -> dict | None:
    # Strategy 1: Look for ```json ... ``` fenced blocks
    # Strategy 2: Find the first { and last } in the text
    # Strategy 3: Try parsing the whole text as JSON directly

    # Example AI response:
    # "Here is the analysis:\n```json\n{\"purpose\": \"Parses JSON\"}\n```"
    # → returns {"purpose": "Parses JSON"}
```

M4 — Code Analysis Engine

`analysis/analysis_types.py` — Data Structures

These are the result objects that flow through the whole pipeline:

```
@dataclass(frozen=True)    # frozen = immutable after creation
class FunctionInfo:
    name: str              # "parse_json"
    line_start: int        # 42
    line_end: int          # 67
    params: list[str]      # ["text", "strict", "*args"]
    complexity: int        # 3  (cyclomatic complexity)
    has_docstring: bool    # True
    calls: list[str]       # ["json.loads", "strip", "validate"]

@dataclass(frozen=True)
class FileAnalysis:
    path: str              # "/home/user/project/src/parser.py"
    language: str          # "python"
    lines_of_code: int     # 156
    functions: list[FunctionInfo]
    classes: list[str]     # ["JSONParser", "ParseError"]
    imports: list[str]     # ["json", "pathlib.Path", "typing.Any"]
    ai_summary: str        # "Parses and validates JSON responses from APIs."
    anti_patterns: list[str]  # ["Missing type annotations on dump()"]
    analyzed_at: float     # 1718640000.0  (Unix timestamp)
```

Both support `.to_dict()` / `.from_dict()` / `.to_json()` / `.from_json()` for serializing over the EventBus and to disk.

`analysis/ast_analyzer.py` — Python AST Parsing

Purpose: Read a `.py` file and extract its structural information WITHOUT running it.

`analyze_python(path, source)` **— detailed walkthrough:**

```
def analyze_python(path, source):
    # Step 1: Count non-empty lines (lines of code)
    loc = sum(1 for line in source.splitlines() if line.strip())

    # Step 2: Parse source into an AST tree
    tree = ast.parse(source)
    # If SyntaxError → return partial result with empty lists

    # Step 3: Walk TOP-LEVEL nodes only
    for node in tree.body:
        if isinstance(node, ast.ClassDef):
            classes.append(node.name)           # "JSONParser"

        elif isinstance(node, ast.FunctionDef):
            functions.append(_function_info(node))

        elif isinstance(node, ast.Import):
            imports.extend(...)                  # "json", "pathlib"

        elif isinstance(node, ast.ImportFrom):
            imports.extend(...)                  # "typing.Any", "typing.Optional"

    # Step 4: Walk ALL nodes (including nested) for inner functions
    for node in ast.walk(tree):
        if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
            if node not in tree.body:  # skip top-level (already found)
                functions.append(_function_info(node))
```

`_function_info(node)` **— extracts:**

```
def _function_info(node):
    # Parameter names (handles *args, **kwargs, keyword-only)
    params = [a.arg for a in node.args.args]  # ["self", "text", "strict"]

    # Function calls inside this function
    calls = _collect_calls(node)  # ["json.loads", "strip"]

    # Has docstring?
    has_doc = ast.get_docstring(node) is not None

    # Line range
    line_start, line_end = node.lineno, node.end_lineno

    # Cyclomatic complexity (see complexity.py)
    complexity = cyclomatic_complexity(node)

    return FunctionInfo(name, line_start, line_end, params, complexity, has_doc, calls)
```

`analysis/complexity.py` — Cyclomatic Complexity

Purpose: Calculate a number representing how "branchy" a function is.

The algorithm: Start at 1, add 1 for each decision point:

Node type	Why it adds complexity
`if / elif`	Branch in control flow
`for / while`	Loop = possible skip
`except`	Error handling branch
`assert`	Can fail
`and / or`	Short-circuit evaluation
`with`	Context manager can raise

```
def cyclomatic_complexity(node: ast.AST) -> int:
    complexity = 1  # base: linear path
    for child in ast.walk(node):
        if isinstance(child, (ast.If, ast.While, ast.For,
                              ast.ExceptHandler, ast.With,
                              ast.Assert)):
            complexity += 1
        elif isinstance(child, ast.BoolOp):
            complexity += len(child.values) - 1  # A and B and C = 2 extra
    return complexity
```

Example dry run:

```
def process(items):      # complexity = 1
    for item in items:   # +1 = 2
        if item > 0:     # +1 = 3
            try:
                save(item)
            except IOError:  # +1 = 4
                log()
    return True          # total = 4
```

`analysis/dependency_mapper.py` — Local Import Detection

Purpose: Find which FILES within the same project a given file imports (not standard library or third-party packages).

```
def resolve_local_import(module_name: str, project_root: Path) -> Path | None:
    # Convert "analysis.complexity" → "analysis/complexity.py"
    parts = module_name.split(".")
    candidate = project_root / Path(*parts)

    # Try as a module file: analysis/complexity.py
    if (candidate.with_suffix(".py")).exists():
        return candidate.with_suffix(".py")

    # Try as a package: analysis/__init__.py
```

```
    if (candidate / "__init__.py").exists():
        return candidate / "__init__.py"

    return None  # not a local import → skip
```

`analysis/analyzer_engine.py` — The Analysis Service

Purpose: The EventBus service that glues everything together for M4.

```
Subscribes to: "watcher.file_change"
Publishes:     "analysis.file_analyzed"
```

```
class Module4AnalyzerEngine(Analyzer):
    def start(self):
        # Subscribe to file changes
        self._bus.subscribe("watcher.file_change", self._on_file_change)
        # Start worker thread that processes the queue
        self._worker = Thread(target=self._worker_loop)
        self._worker.start()

    def _on_file_change(self, payload):
        event = payload["event"]
        if event.kind != ChangeKind.DELETED:
            self._queue.put(event.path)  # add to work queue

    def _worker_loop(self):
        while not self._stop.is_set():
            try:
                path = self._queue.get(timeout=0.25)
                analysis = analyze_python_file(path)  # the actual analysis
                self._bus.publish("analysis.file_analyzed", {
                    "file_path": str(path),
                    "analysis": analysis
                })
            except queue.Empty:
                continue
```

M5 — Documentation Engine

`docs/frontmatter.py` — YAML Header Builder

Every generated note starts with a YAML frontmatter block:

```
---
file: src/parser.py
language: python
lines: 156
complexity: 3.4
last_analyzed: 2026-06-17T21:00:00
```

```
tags: [python, src, parser, projectmind]
---
```

```
def build_frontmatter(analysis: FileAnalysis) -> str:
    tags = _infer_tags(analysis)   # from language, directory, filename
    complexity = _weighted_complexity(analysis)
    last_analyzed = datetime.fromtimestamp(analysis.analyzed_at).isoformat()
    # Build YAML string using PyYAML
    data = {"file": analysis.path, "language": ..., "lines": ..., ...}
    return f"---\n{yaml.dump(data)}---\n"
```

`docs/doc_generator.py` — Full Document Assembly

Purpose: Take a `FileAnalysis` and produce a complete markdown document.

Output structure:

```
---
(frontmatter)
---

# parser.py

> AI-written summary of what this file does.

## Functions

| Name | Params | Complexity | Docstring? |
|------|--------|------------|------------|
| `parse` | text, strict | 4 | ✓ |
| `dump` | obj | 1 | ✗ |

## Anti-Patterns

- Missing type annotations on dump()

## Dependencies

- `json`
- `pathlib.Path`

## Changelog

- **[FUNCTION_ADDED]** Function `parse` added _2026-06-17 21:00_
```

The document is built section by section using Jinja2 templates:

```
def generate(analysis: FileAnalysis, changelog_entries=None) -> str:
    fm = build_frontmatter(analysis)
    body = render_doc_template("file_doc", {
        "analysis": analysis,
        "functions": analysis.functions,
        "anti_patterns": analysis.anti_patterns,
        "changelog": format_changelog(changelog_entries or [])
    })
    return fm + body
```

`docs/changelog.py` — Change Detection

Purpose: Compare two `FileAnalysis` objects and report what changed.

```
def diff_analyses(old: FileAnalysis, new: FileAnalysis) -> list[ChangelogEntry]:
    entries = []

    # Were any functions added?
    old_names = {f.name for f in old.functions}
    new_names = {f.name for f in new.functions}
    for name in new_names - old_names:
        entries.append(ChangelogEntry("FUNCTION_ADDED", name, timestamp))

    # Were any functions removed?
    for name in old_names - new_names:
        entries.append(ChangelogEntry("FUNCTION_REMOVED", name, timestamp))

    # Did complexity change?
    if abs(old_complexity - new_complexity) > 0.1:
        entries.append(ChangelogEntry("COMPLEXITY_CHANGED", ...))

    # Did imports change?
    if set(old.imports) != set(new.imports):
        entries.append(ChangelogEntry("IMPORTS_CHANGED", ...))

    return entries
```

M6 — Graph Engine

`graph/graph_builder.py` — networkx Wrapper

Purpose: Maintain a directed graph where nodes are files and edges are imports.

```
node: "src/parser.py"   → attributes: {language, complexity, function_count, last_analyzed}
edge: "src/main.py" → "src/parser.py"  means: main.py imports parser.py
```

```
class GraphEngine:
    def __init__(self):
        self.graph = nx.DiGraph()  # directed graph from networkx

    def update_node(self, analysis: FileAnalysis):
        # Add or update the node for this file
        self.graph.add_node(analysis.path,
            language=analysis.language,
            complexity=_weighted_complexity(analysis),
            function_count=len(analysis.functions),
            last_analyzed=analysis.analyzed_at
        )
```

```
    def update_edges(self, analysis: FileAnalysis) -> tuple[list, list]:
        # Find which local files this file imports
        new_targets = resolve_imports(analysis)
        old_targets = set(self.graph.successors(analysis.path))

        # Add new edges
        added = [t for t in new_targets if t not in old_targets]
        for t in added:
            self.graph.add_edge(analysis.path, t)

        # Remove stale edges
        removed = [t for t in old_targets if t not in new_targets]
        for t in removed:
            self.graph.remove_edge(analysis.path, t)

        return added, removed

    def get_related_files(self, path: str, depth=2) -> list[str]:
        # BFS up to 'depth' hops from 'path'
        return list(nx.bfs_tree(self.graph, path, depth_limit=depth).nodes)
```

`graph/graph_analyzer.py` — Graph Analysis

Pure functions (no side effects, no state):

```
def find_orphans(graph) -> list[str]:
    # Nodes with NO incoming OR outgoing edges
    # (files that nobody imports and that import nobody)
    return [n for n in graph.nodes
            if graph.in_degree(n) == 0 and graph.out_degree(n) == 0]

def find_hubs(graph, threshold=5) -> list[str]:
    # Nodes imported by many others (high in-degree)
    # These are the most "critical" files — breaking them breaks everything
    return sorted([n for n in graph.nodes if graph.in_degree(n) >= threshold],
                  key=lambda n: graph.in_degree(n), reverse=True)

def find_circular_deps(graph) -> list[list[str]]:
    # Find all cycles using Johnson's algorithm
    # A cycle means: A imports B imports C imports A (BAD!)
    return list(nx.simple_cycles(graph))

def complexity_hotspots(graph) -> list[str]:
    # Files that are BOTH highly complex AND heavily imported
    # These are the most dangerous files to modify
    top_complexity = top_quartile(complexities)
    top_in_degree = top_quartile(in_degrees)
    return [n for n in graph.nodes
            if complexity[n] >= top_complexity and in_degree[n] >= top_in_degree]
```

`graph/graph_state.py` — Persistence

Purpose: Save/load the graph to disk so it survives restarts.

```
def save_graph(graph, path):
    # Convert graph to JSON-serializable format
    data = nx.node_link_data(graph)  # {"nodes": [...], "links": [...]}

    # Atomic write: write to .tmp file first, then rename
    tmp = path.with_suffix(".tmp")
    tmp.write_text(json.dumps(data))
    tmp.rename(path)  # atomic on POSIX systems

def load_graph(path) -> DiGraph:
    if not path.exists():
        return nx.DiGraph()  # fresh empty graph
    try:
        data = json.loads(path.read_text())
        return nx.node_link_graph(data)
    except Exception:
        log.warning("Corrupt graph state — starting fresh")
        return nx.DiGraph()

def record_update(graph) -> bool:
    self._counter += 1
    if self._counter >= self._auto_save_interval:  # every 10 updates
        save_graph(graph, self._path)
        self._counter = 0
        return True
    return False
```

Why atomic write? If the program crashes mid-write, the old file is untouched (`.tmp` was never renamed). Without this, a crash could leave a half-written corrupt file.

M7 — Memory Engine

`memory/chunker.py` — Text Chunking

Purpose: Break a `FileAnalysis` into small overlapping text segments ("chunks") suitable for embedding. Smaller chunks = more precise search results.

```
def chunk_analysis(analysis: FileAnalysis) -> list[MemoryChunk]:
    chunks = []

    # Chunk 1: File-level overview
    text = f"File: {analysis.path}\nSummary: {analysis.ai_summary}\n"
    text += f"Classes: {', '.join(analysis.classes)}"
    chunks.append(MemoryChunk(
        id=f"{analysis.path}::overview",
        text=text,
        metadata={"path": analysis.path, "chunk_type": "overview"}
    ))

    # One chunk per function
    for fn in analysis.functions:
        text = f"Function {fn.name} in {analysis.path}\n"
        text += f"Parameters: {', '.join(fn.params)}\n"
        text += f"Complexity: {fn.complexity}"
```

```
        chunks.append(MemoryChunk(
            id=f"{analysis.path}::{fn.name}",
            text=text,
            metadata={"path": ..., "chunk_type": "function", "function": fn.name}
        ))

    return chunks
```

`memory/embedder.py` — Text-to-Vector Conversion

Purpose: Convert text into a list of numbers (embedding vector) using sentence-transformers. Similar texts → similar vectors.

```
class Embedder:
    def __init__(self, model_name="all-MiniLM-L6-v2"):
        # Load the sentence-transformer model (downloads first time ~90MB)
        from sentence_transformers import SentenceTransformer
        self._model = SentenceTransformer(model_name)

    def encode(self, text: str) -> list[float]:
        # Returns a list of 384 floats
        return self._model.encode(text).tolist()

    def batch_encode(self, texts: list[str]) -> list[list[float]]:
        # More efficient than calling encode() in a loop
        return self._model.encode(texts).tolist()
```

Example: `"Parse JSON response"` → `[0.12, -0.45, 0.78, ...(384 values total)]`

`memory/memory_store.py` — ChromaDB Wrapper

Purpose: Store and search vector embeddings using ChromaDB (an embedded vector database).

```
class MemoryStore:
    def __init__(self, persist_directory: str):
        self._client = chromadb.PersistentClient(path=persist_directory)
        self._collection = self._client.get_or_create_collection("projectmind")

    def upsert(self, chunks: list[MemoryChunk], embeddings: list[list[float]]):
        # "upsert" = insert if new, update if exists (based on chunk.id)
        self._collection.upsert(
            ids=[c.id for c in chunks],
            documents=[c.text for c in chunks],
            embeddings=embeddings,
            metadatas=[c.metadata for c in chunks]
        )

    def search(self, query_embedding: list[float], top_k=5) -> list[SearchResult]:
        results = self._collection.query(
            query_embeddings=[query_embedding],
            n_results=top_k
        )
        # Returns the 'top_k' most similar chunks
```

```
        return parse_results(results)

    def delete_by_path(self, path: str):
        # Remove all chunks for a deleted file
        self._collection.delete(where={"path": path})
```

`memory/semantic_search.py` — Query Interface

Purpose: The public API for searching the memory. Combines embedding + ChromaDB search.

```
class SemanticSearch:
    def __init__(self, store: MemoryStore, embedder: Embedder):
        self._store = store
        self._embedder = embedder

    def search(self, query: str, top_k: int = 5) -> list[SearchResult]:
        # Step 1: Convert query text to embedding vector
        query_embedding = self._embedder.encode(query)

        # Step 2: Find the closest chunks in vector space
        return self._store.search(query_embedding, top_k=top_k)
```

Example:

```
results = search.search("function that parses JSON responses")
# Returns the top 5 most semantically similar code chunks
# even if none of them contain the exact words "parse", "JSON", "responses"
```

M8 — Obsidian Engine

`obsidian/vault.py` — VaultManager

Purpose: Read and write markdown notes to the Obsidian vault directory.

```
class VaultManager:
    def write_note(self, section, name, body, frontmatter_extras=None) -> Path:
        # Builds path: vault/Generated/parser.md
        path = self._root / section / f"{name}.md"
        path.parent.mkdir(parents=True, exist_ok=True)

        # Merge frontmatter defaults with extras
        fm = {**self._defaults, **(frontmatter_extras or {})}
        content = f"---\n{yaml.dump(fm)}---\n\n{body}"

        # Atomic write: .tmp → rename
        tmp = path.with_suffix(".tmp")
        tmp.write_text(content, encoding="utf-8")
        tmp.rename(path)
        return path
```

```
    def read_note(self, section, name) -> tuple[dict, str]:
        # Returns (frontmatter_dict, body_string)
        content = (self._root / section / f"{name}.md").read_text()
        fm, body = split_frontmatter(content)
        return fm, body
```

`obsidian/vault_index.py` — In-Memory Index

Purpose: Track all notes in the vault by their "stem" (filename without `.md`)

for fast wikilink resolution.

```
class VaultIndex:
    def __init__(self, vault_root: Path):
        self._index: dict[str, Path] = {}  # "parser" → vault/Generated/parser.md

    def startup_scan(self):
        # Walk the whole vault and index every .md file
        for path in self._vault_root.rglob("*.md"):
            self._index[path.stem] = path

    def find_note(self, stem: str) -> Path | None:
        return self._index.get(stem)

    def update(self, event_kind, path):
        if event_kind == "write":
            self._index[path.stem] = path
        elif event_kind == "delete":
            self._index.pop(path.stem, None)
```

`obsidian/link_resolver.py` — Wikilink Creation

Purpose: Convert file paths to `[[wikilinks]]`, handling duplicate filenames.

```
def path_to_wikilink(path: Path, vault_root: Path) -> str:
    # If the path is inside the vault, use relative path
    try:
        relative = path.relative_to(vault_root)
        stem = relative.stem   # "parser"
    except ValueError:
        stem = path.stem       # outside vault → just use filename

    return f"[[{stem}]]"

def resolve_links(paths: list[Path], vault_root: Path) -> list[str]:
    stems = [p.stem for p in paths]
    # Detect duplicates
    duplicates = {s for s in stems if stems.count(s) > 1}

    result = []
    for path in paths:
        if path.stem in duplicates:
            # Disambiguate: [[subdir/parser]] instead of [[parser]]
```

```
            try:
                relative = path.relative_to(vault_root)
                result.append(f"[[{relative.with_suffix('')}]]")
            except ValueError:
                result.append(f"[[{path.stem}]]")
        else:
            result.append(f"[[{path.stem}]]")
    return result
```

`obsidian/note_builder.py` — Final Note Assembly

Purpose: Combine the doc content with graph links and semantic neighbours.

```
class NoteBuilder:
    def build(self, path, markdown_content, graph_links, semantic_neighbours) -> str:
        parts = [markdown_content]  # Start with the doc content

        # Add related files section (from graph)
        if graph_links:
            links = self._resolver.resolve_links(graph_links, self._vault_root)
            parts.append("\n## Related Files\n")
            parts.extend(f"- {link}\n" for link in links)

        # Add semantic neighbours section (from ChromaDB)
        if semantic_neighbours:
            parts.append("\n## Semantic Neighbours\n")
            for result in semantic_neighbours[:3]:  # top 3 only
                parts.append(f"- [[{Path(result.path).stem}]] — {result.score:.2f}\n")

        return "".join(parts) + "\n"
```

`obsidian/vault_writer.py` — Async Write Queue

Purpose: All vault writes go through a queue so they don't block the event processing thread. Writes happen in a dedicated background thread.

```
class VaultWriter:
    def __init__(self, vault_root):
        self._queue = queue.Queue()  # queue of write tasks
        self._thread = None

    def write(self, path: Path, content: str):
        self._queue.put(("write", path, content))  # non-blocking

    def delete(self, path: Path):
        self._queue.put(("delete", path, None))

    def _worker_loop(self):
        while not self._stop.is_set():
            try:
                kind, path, content = self._queue.get(timeout=0.25)
                if kind == "write":
                    path.parent.mkdir(parents=True, exist_ok=True)
```

```
                    tmp = path.with_suffix(".tmp")
                    tmp.write_text(content, encoding="utf-8")
                    tmp.rename(path)  # atomic
                elif kind == "delete":
                    path.unlink(missing_ok=True)
            except queue.Empty:
                continue
```

M9 — Git Integration Engine

`git_integration/git_types.py` — Data Structures

```
@dataclass
class CommitInfo:
    hash: str          # "abc1234"
    author: str        # "Alice <alice@example.com>"
    date: str          # "2026-06-17T21:00:00+05:30"
    message: str       # "Fix JSON parsing edge case"
    diff: str          # the full diff text

@dataclass
class CommitSummary:
    commit_hash: str
    summary: str       # "Fixed a bug where empty strings caused JSON parse failure"
    impact: str        # "low" | "medium" | "high"
    files_changed: list[str]
    key_changes: list[str]
```

`git_integration/git_monitor.py` — Commit Detection

Purpose: Poll the git repository for new commits and publish them to the EventBus.

```
class GitMonitor:
    def __init__(self, repo_path, bus, poll_interval):
        self._repo_path = repo_path
        self._bus = bus
        self._poll_interval = poll_interval  # e.g., 60 seconds
        self._last_seen_hash = None          # remember what we've already processed

    def _get_new_commits(self) -> list[CommitInfo]:
        # Run: git log --since=<last_check> --format="%H|%ae|%ci|%s" HEAD
        result = subprocess.run(
            ["git", "log", "--oneline", "--since", self._last_check],
            capture_output=True, text=True, cwd=self._repo_path
        )
        return parse_commit_lines(result.stdout)

    def _poll_loop(self):
        while not self._stop.is_set():
            commits = self._get_new_commits()
            for commit in commits:
```

```
                self._bus.publish("git.commit", {"commit": commit})
                self._last_seen_hash = commit.hash
            time.sleep(self._poll_interval)
```

`git_integration/commit_summarizer.py` — AI-Powered Summaries

Purpose: For each new commit, ask the AI to produce a structured summary.

```
class CommitSummarizer:
    def summarize(self, commit: CommitInfo) -> CommitSummary:
        # Ask AI: "Given this diff, what changed and why?"
        text = get_ai().complete("commit_summary", {
            "commit_hash": commit.hash,
            "commit_message": commit.message,
            "diff": commit.diff[:4000],  # truncate huge diffs
        })

        # Parse the JSON response
        data = parse_json_object(text)
        return CommitSummary(
            commit_hash=commit.hash,
            summary=data.get("summary", ""),
            impact=data.get("impact", "low"),
            files_changed=data.get("files_changed", []),
            key_changes=data.get("key_changes", []),
        )
```

M10 — Intelligence Engine

`intelligence/pattern_detector.py` — Anti-Pattern Detection

Purpose: Analyze the graph and analysis data to find structural problems.

```
def detect_anti_patterns(graph, analyses) -> list[Pattern]:
    patterns = []

    # Pattern 1: Circular dependencies
    cycles = find_circular_deps(graph)
    for cycle in cycles:
        patterns.append(Pattern(
            kind="CIRCULAR_DEPENDENCY",
            severity="high",
            affected_files=cycle,
            description=f"Circular import: {' → '.join(cycle)}"
        ))

    # Pattern 2: God files (very high complexity + many imports)
    for analysis in analyses:
        if analysis.lines_of_code > 500 and len(analysis.functions) > 20:
            patterns.append(Pattern(
                kind="GOD_FILE",
```

```
                severity="medium",
                affected_files=[analysis.path],
                description=f"{analysis.path} has {len(analysis.functions)} functions"
            ))

    # Pattern 3: Orphan files (nothing imports them, they import nothing)
    orphans = find_orphans(graph)
    for path in orphans:
        patterns.append(Pattern(kind="ORPHAN_FILE", severity="low", ...))

    # Pattern 4: Hub files (imported by many → high blast radius)
    hubs = find_hubs(graph, threshold=5)
    for path in hubs:
        patterns.append(Pattern(kind="HUB_FILE", severity="medium", ...))

    return patterns
```

`intelligence/refactor_suggester.py` — AI Refactor Suggestions

Purpose: For each detected pattern, ask AI for a concrete fix.

```
def suggest(pattern: Pattern, ai) -> Suggestion:
    text = ai.complete("refactor_suggestion", {
        "pattern_kind": pattern.kind,
        "severity": pattern.severity,
        "affected_files": pattern.affected_files,
        "description": pattern.description,
    })
    data = parse_json_object(text)
    return Suggestion(
        pattern=pattern,
        suggestion=data.get("suggestion", ""),
        rationale=data.get("rationale", ""),
        effort=data.get("effort", "medium"),
        status="PENDING",
    )
```

`intelligence/suggestion_store.py` — Persistent Deduplication

Purpose: Save suggestions to disk and avoid creating duplicate suggestions

for the same problem.

```
class SuggestionStore:
    def get_pending(self, pattern_fingerprint: str) -> Suggestion | None:
        # Check if we already have a PENDING suggestion for this exact pattern
        for suggestion in self._suggestions:
            if (suggestion.status == "PENDING"
                    and suggestion.fingerprint == pattern_fingerprint):
                return suggestion
        return None

    def add(self, suggestion: Suggestion):
        self._suggestions.append(suggestion)
```

```
        self._save()  # persist to disk immediately

    def _fingerprint(self, pattern: Pattern) -> str:
        # Create a stable string that identifies a pattern uniquely
        return hashlib.md5(
            f"{pattern.kind}:{':'.join(sorted(pattern.affected_files))}".encode()
        ).hexdigest()
```

`intelligence/intelligence_engine.py` — The Intelligence Service

```
Subscribes to: "graph.graph_updated"
Publishes:     "intelligence.suggestions_ready"
Runs cycle:    every 60 minutes (or immediately on graph update)
```

```
class IntelligenceEngine:
    def _run_cycle(self):
        # Step 1: Detect patterns in the current graph + analyses
        patterns = detect_anti_patterns(self._graph, self._analyses)

        # Step 2: For each pattern, check if we already have a suggestion
        new_suggestions = []
        for pattern in patterns:
            fingerprint = self._store._fingerprint(pattern)
            if self._store.get_pending(fingerprint) is None:
                # Step 3: Generate a new suggestion via AI
                suggestion = suggest(pattern, get_ai())
                self._store.add(suggestion)
                new_suggestions.append(suggestion)

        # Step 4: Publish if any new suggestions were generated
        if new_suggestions:
            self._bus.publish("intelligence.suggestions_ready", {
                "suggestions": [s.__dict__ for s in new_suggestions]
            })

    def query_codebase(self, question: str) -> str:
        # Step 1: Search memory for relevant chunks
        results = self._memory.search(question, top_k=5)
        context = "\n\n".join(r.text for r in results)

        # Step 2: Ask AI with the context
        return get_ai().complete_raw(
            f"Context from codebase:\n{context}\n\nQuestion: {question}"
        )
```

---

6. Working Flow — End to End

The Full Chain of Events

Here is what happens from the moment you save a Python file until a note appears in your Obsidian vault:

```
You save "src/parser.py"
       │
       ▼
[M2 — WatcherManager]
watchdog detects MODIFIED event
→ FileTracker records it, starts debounce timer
→ After 2 seconds of quiet: bus.publish("watcher.file_change", {...})
       │
       ▼
[M3 — AIManager]  (subscribed to watcher.file_change)
→ logs the event (currently just observes, future: trigger AI on demand)
       │
       ▼
[M4 — Module4AnalyzerEngine]  (subscribed to watcher.file_change)
→ puts "src/parser.py" in the work queue
→ worker thread picks it up
→ ast_analyzer.analyze_python_file("src/parser.py"):
     1. Read file contents
     2. ast.parse() → build AST tree
     3. Walk AST → extract functions, classes, imports
     4. Calculate cyclomatic complexity for each function
     5. Call get_ai().complete("code_analysis", {...})
        → Ollama returns {"purpose": "...", "suggestions": [...]}
     6. Package everything into FileAnalysis object
→ bus.publish("analysis.file_analyzed", {"file_path": ..., "analysis": FileAnalysis})
       │
       ┌────────────────────────────────────────────────┐
       ▼                                                ▼
[M5 — Module5DocEngine]              [M6 — Module6GraphEngine]
(subscribed to analysis.file_analyzed) (subscribed to analysis.file_analyzed)

→ build_frontmatter(analysis)          → update_node(analysis)  ← add/update node
→ generate(analysis, changelog)        → update_edges(analysis) ← sync import edges
→ bus.publish("docs.doc_updated", {    → record_update() → auto-save every 10 updates
    "path": ...,                      → bus.publish("graph.graph_updated", {
    "markdown_content": ...,              "updated_node": ...,
    "frontmatter": ...                    "edges_added": [...],
  })                                      "stats": {...}
       │                                })
       │                                   │
       ▼                                   │
[M7 — MemoryUpdater]               [M10 — IntelligenceEngine]
(subscribed to analysis.file_analyzed) (subscribed to graph.graph_updated)
→ chunk_analysis(analysis)         → queues a cycle run
→ embedder.batch_encode(chunks)    → detect_anti_patterns(graph, analyses)
→ memory_store.upsert(chunks, emb) → for new patterns: suggest() via AI
                                  → store.add(suggestion)
                                  → bus.publish("intelligence.suggestions_ready")
       │
       ▼
[M8 — ObsidianEngine]
(subscribed to docs.doc_updated AND graph.graph_updated)

On docs.doc_updated:
→ get graph links for this file (related files, depth=2)
→ search semantic memory for nearest neighbours
→ note_builder.build(markdown, graph_links, neighbours)
→ vault_writer.write(vault/Generated/parser.md, final_content)
→ vault_index.update("write", path)
→ bus.publish("obsidian.note_written", {...})
```

Final result: `vault/Generated/parser.md` now contains:

- YAML frontmatter with metadata
- AI summary
- Functions table with complexity scores
- Anti-patterns list
- Dependencies list
- Changelog (what changed vs last analysis)
- Related Files section with `[[wikilinks]]`
- Semantic Neighbours section with similarity scores

Git Flow

```
Every 60 seconds:
       ■
[M9 — GitMonitor]
→ git log --since=<last_check>
→ for each new commit:
    bus.publish("git.commit", {"commit": CommitInfo})
          ■
          ▼
[M9 — CommitSummarizer] (inside GitEngine)
→ get_ai().complete("commit_summary", {diff, message})
→ parse JSON response → CommitSummary
→ git_memory.store(summary)  ← stores in ChromaDB
→ bus.publish("git.commit_summarized", {...})
```

---

7. Algorithms and Logic

Algorithm 1 — Cyclomatic Complexity

Input: An `ast.FunctionDef` node

Output: An integer ≥ 1

Rule: Start at 1. Add 1 for each `if`, `elif`, `for`, `while`, `except`, `assert`, `with`. For `and`/`or`, add `len(values) - 1`.

Dry run:

```
def example(data, flag):     # start: complexity = 1
    if not data:             # if → +1 = 2
        return None
```

```
    for item in data:        # for → +1 = 3
        if flag and item > 0:  # if → +1 = 4,  and → +1 = 5
            try:
                process(item)
            except ValueError:  # except → +1 = 6
                log_error()
    return True              # FINAL complexity = 6
```

Algorithm 2 — Debounce (File Tracker)

Problem: A file save triggers 10 events in 0.1 seconds. We want exactly 1 analysis.

Algorithm:

```
For each raw event:
    record(path → latest_event, path → timestamp)

Every 100ms (flush loop):
    for each pending path:
        age = current_time - last_seen_time[path]
        if age >= debounce_seconds (e.g., 2.0):
            emit the event
            remove from pending
```

Dry run:

```
T=0.00  MODIFIED parser.py   → pending: {parser.py: T=0.00}
T=0.05  MODIFIED parser.py   → pending: {parser.py: T=0.05}  ← timestamp updated
T=0.10  MODIFIED parser.py   → pending: {parser.py: T=0.10}
...
T=2.15  flush loop runs:
        age = 2.15 - 0.10 = 2.05 ≥ 2.0 → EMIT! → analysis starts
```

Algorithm 3 — Graph Cycle Detection (Johnson's Algorithm)

Problem: Find all circular imports in the dependency graph.

Concept: networkx's `simple_cycles()` uses Johnson's algorithm. It finds all **elementary circuits** (cycles where no node repeats).

Example:

```
Nodes: A, B, C, D
Edges: A→B, B→C, C→A, B→D, D→B

Cycles found:
```

```
  [A, B, C]   ← A imports B, B imports C, C imports A (BAD!)
  [B, D]      ← B imports D, D imports B (BAD!)
```

These are reported as patterns with severity="high" by the Intelligence Engine.

Algorithm 4 — Semantic Search (Vector Space)

Step 1: Convert query to vector

```
query = "function that validates input data"
query_vector = embedder.encode(query)  # [0.12, -0.45, 0.78, ...]
```

Step 2: ChromaDB computes cosine distance to all stored vectors

```
cosine_similarity(query_vector, chunk_vector) =
    dot_product(query_vector, chunk_vector) /
    (magnitude(query_vector) * magnitude(chunk_vector))

→ Returns a score from -1 (opposite) to 1 (identical)
```

Step 3: Return top-K highest-similarity chunks

Why cosine similarity? It measures the *angle* between vectors, not their magnitude. "Validate input" and "check if input is valid" point in similar directions in embedding space even though they share few exact words.

Algorithm 5 — Deduplication (Suggestion Store)

Problem: The intelligence engine runs every hour. We don't want 24 identical suggestions for the same circular import.

Solution: MD5 fingerprint of `(pattern_kind, sorted_affected_files)`:

```
fingerprint = md5(f"CIRCULAR_DEPENDENCY:analysis/ast.py:analysis/complexity.py")

# Before generating a new suggestion:
if store.get_pending(fingerprint):
    skip  # already have one
else:
    generate_and_store()
```

---

8. Database and API Integration

ChromaDB — Vector Database

What it is: An embedded database (runs inside your Python process, no separate server) that stores text + embeddings and supports fast similarity search.

How ProjectMind uses it:

```
Storage:    memory_store.upsert(chunks, embeddings)
            → stored in .chroma/ directory (persistent on disk)

Retrieval:  memory_store.search(query_embedding, top_k=5)
            → returns 5 most similar chunks with their metadata

Deletion:   memory_store.delete_by_path("src/old_file.py")
            → removes all chunks for a deleted file
```

ChromaDB Collection: Think of a collection like a database table. ProjectMind uses one collection called `"projectmind"` that holds ALL chunks from ALL files. Each chunk has an `id` (unique), `document` (text), `embedding` (vector), and `metadata` (dict with path, chunk_type, etc.).

Ollama API — Local AI Server

What it is: A local HTTP server at `http://localhost:11434` that runs AI models.

Endpoints used:

Endpoint	Purpose
`POST /api/chat`	Send a system+user message and get a response
`POST /api/generate`	Send a raw prompt (no chat format)
`GET /api/tags`	List available models (used for availability check)

Request format:

```
client.chat(
    model="qwen2.5-coder:14b",
    messages=[
        {"role": "system", "content": "You are an expert code reviewer..."},
        {"role": "user",   "content": "Analyze this Python file: ..."},
    ],
    options={"temperature": 0.2, "num_predict": 4096}
)
```

Response:

```
response.message.content  # → "This file implements JSON parsing..."
response.model            # → "qwen2.5-coder:14b"
response.eval_count       # → 312  (tokens generated)
```

Git via subprocess

How ProjectMind reads git history:

```
# Get commit hashes since last check
result = subprocess.run(
    ["git", "log", "--format=%H", "--since", "2 hours ago"],
    capture_output=True, text=True, cwd=repo_path
)
hashes = result.stdout.strip().split("\n")

# Get full commit info
result = subprocess.run(
    ["git", "show", "--stat", hash],
    capture_output=True, text=True, cwd=repo_path
)
```

No gitpython library is used — plain subprocess calls to the `git` command. This keeps the dependency simple and avoids version compatibility issues.

---

9. Error Handling

Philosophy: "Be loud about crashes, silent about non-critical failures"

ProjectMind uses a layered approach:

Layer 1 — Fatal errors stop the app

```
# In main.py:
try:
    app = bootstrap()
except ProjectMindError as exc:
    print(f"[FATAL] {exc}", file=sys.stderr)
    return 1  # exit with error code
```

If bootstrap fails (bad config, Ollama unreachable), the whole app exits cleanly.

Layer 2 — Service errors are logged, not raised

```
# In the worker loops:
def _worker_loop(self):
    while not self._stop.is_set():
        try:
            payload = self._queue.get(timeout=0.25)
            self._process(payload)
        except queue.Empty:
            continue
        except Exception as exc:
            log.exception("[graph] error processing analysis: %s", exc)
            # Does NOT re-raise — the worker loop continues!
```

If one file analysis fails (e.g., the AI times out), the next file still gets processed.

Layer 3 — AI failures never break core logic

```
# In ast_analyzer.py:
try:
    text = get_ai().complete("code_analysis", {...})
    parsed = parse_json_object(text)
    ai_summary = parsed.get("purpose", "")
except Exception:  # catches ALL exceptions
    pass  # AI failure = empty summary. Analysis still proceeds.
```

The AST analysis (structure, functions, imports, complexity) always works even if the AI is down. You just get empty `ai_summary` and `anti_patterns` fields.

Layer 4 — Shutdown hooks are bullet-proof

```
def shutdown(self):
    while self._shutdown_hooks:
        hook = self._shutdown_hooks.pop()
        try:
            hook()          # call service.stop()
        except Exception:
            log.exception("Shutdown hook %r failed", hook)
            # CONTINUE — next hook must still run!
```

One service failing to stop does not prevent other services from stopping.

Layer 5 — Atomic file writes prevent corruption

```
# Instead of directly writing:
path.write_text(content)  # DANGEROUS: crash mid-write = corrupt file
```

```
# ProjectMind always does:
tmp = path.with_suffix(".tmp")
tmp.write_text(content)   # write to temp file
tmp.rename(path)           # atomic on POSIX — either works or doesn't
```

Error Table

Situation	Exception	Handler
Config YAML is malformed	`ConfigError`	`bootstrap()` catches → `BootstrapError` → `main()` exits
Service key not in registry	`RegistryError`	Caller must handle; bootstrap usually exits
Vault directory not writable	`VaultError`	Logged; specific note write fails silently
Ollama server down	`AIError`	`main()` logs + exits if at startup; analysis proceeds with empty AI fields
Model not found in Ollama	`AIError`	`_should_fallback()` → retry with fallback model
AI response not valid JSON	`ResponseParseError`	Caught in `analyze_python_file()` → empty AI fields
File read fails (OSError)	Caught in `analyze_python_file()`	Returns minimal FileAnalysis
Graph state file corrupted	Caught in `load_graph()`	Fresh empty graph; logs warning
ChromaDB write fails	`MemoryError`	Logged; chunk is skipped

10. Interview Preparation

Questions and Answers

Q1: What is ProjectMind and what problem does it solve?

A: ProjectMind is an autonomous AI-powered documentation and knowledge management system for codebases. It solves the problem that documentation is always out of date, codebases are hard to navigate, and developers waste time answering questions that could be answered automatically. It watches your code, analyzes every file, generates docs, builds a dependency graph, stores semantic memory, monitors git history, and

proactively suggests improvements.

Q2: What is an EventBus and why is it used here?

A: An EventBus is a publish-subscribe (pub/sub) messaging system. Instead of Module A directly calling Module B, A publishes a named event to the bus, and any module that has subscribed to that event gets called. This decouples the modules — they don't import each other, which prevents circular imports and makes the system modular. For example, the Watcher doesn't know about the Analyzer — it just publishes `watcher.file_change` and whoever cares will handle it.

Q3: What is cyclomatic complexity and how is it calculated?

A: Cyclomatic complexity measures how many independent execution paths exist in a function. Start at 1 (the straight-line path), then add 1 for each `if`, `for`, `while`, `except`, and each `and`/`or` operator. Higher complexity = harder to test and understand. ProjectMind uses this to identify hotspot functions that need refactoring.

Q4: What is the difference between `complete()` and `complete_raw()`?

A: `complete(prompt_name, variables)` looks up a named template, renders it with variables (using Jinja2), then sends system+user messages to Ollama. `complete_raw(text)` skips the template and sends the raw text directly. `complete()` is for structured, repeatable AI tasks. `complete_raw()` is for one-off queries like the Q&A feature.

Q5: How does the fallback model mechanism work?

A: When calling Ollama with the primary model (`qwen2.5-coder:14b`), if Ollama returns a "model not found" error, `_should_fallback()` checks two conditions: (1) we haven't already tried the fallback, and (2) the error looks like a missing model. If both are true, the same request is retried with `fallback_model`. This allows graceful degradation when a large model isn't available.

Q6: What is semantic search and how does it differ from keyword search?

A: Keyword search finds documents containing exact words. Semantic search finds documents with similar *meaning* by comparing vector embeddings. For example, searching "input validation function" would not find a function documented as "check if data is clean", but semantic search would — because both phrases embed to similar vectors in the mathematical space learned by sentence-transformers.

Q7: Why use atomic file writes (`.tmp` → rename)?

A: If the program crashes while writing a file, you can end up with a partially written file that's unreadable. By writing to a `.tmp` file first and then renaming it, the rename operation is atomic on POSIX systems — it either fully succeeds or leaves the original file intact. There's never a window where a corrupt file exists at the target path.

Q8: What is ChromaDB and why was it chosen?

A: ChromaDB is an embedded vector database — it runs inside the Python process (no separate server needed). It stores text documents alongside their vector embeddings and supports fast approximate nearest-neighbour search. It was chosen because it's simple to set up (just a directory), has a clean Python API, and handles persistence automatically.

Q9: What is the Service Registry pattern and why is it better than direct imports?

A: The Service Registry is a dictionary that maps interface types (or string keys) to live service instances. Instead of `from ai.ai_manager import ai_manager` (which creates circular imports in large codebases), you do `registry.get(AIClient)`. This means: (1) no circular imports, (2) easy to swap implementations in tests (`registry.register(AIClient, FakeAI)`), (3) the wiring is visible in one place (`bootstrap.py`).

Q10: What does "frozen=True" mean in a dataclass?

A: `@dataclass(frozen=True)` makes the dataclass instances immutable — you can't change any field after creation. This is useful for `FileAnalysis` and `FunctionInfo` because they represent analysis results that should never change. It also makes them hashable, so they can be used as dict keys or in sets.

Q11: How does the debounce mechanism work?

A: File editors often trigger multiple save events in rapid succession. Debounce means: don't process an event until the file has been quiet for at least N seconds. The `FileTracker` records the timestamp of the most recent event per file. The flush loop runs every 100ms and only emits an event if `(now - last_seen_time) >= debounce_seconds`. This converts a burst of 10 events into exactly 1 processed event.

Q12: What is the difference between `subscribe()` and `publish()` on the EventBus?

A: `subscribe(event_name, handler)` registers a callback function to be called whenever a specific event fires. `publish(event_name, payload)` fires the event and calls all registered handlers with the payload dict. The EventBus takes a snapshot of handlers before calling them (to avoid issues if a handler unsubscribes during iteration), and all calls are synchronous (the publisher waits for all handlers to finish).

Q13: How does ProjectMind avoid generating the same refactor suggestion twice?

A: The `SuggestionStore` computes an MD5 fingerprint for each pattern using its kind and the sorted list of affected files. Before generating a new suggestion, it checks `get_pending(fingerprint)`. If a suggestion with that fingerprint already has status `PENDING`, it skips generation. This prevents flooding the user with duplicate suggestions across hourly intelligence cycles.

Q14: What happens if Ollama is not running when ProjectMind starts?

A: In `main()`, after bootstrap succeeds, the code calls `ai.start()` which calls `is_available()`. This calls `client.list()` (Ollama's endpoint for listing models). If Ollama is unreachable, `is_available()` returns `False`, `start()` raises `AIError`, and `main()` logs the error and returns exit code 1. The application does not start without a working AI connection.

Q15: What is the LIFO (Last In, First Out) shutdown order and why?

A: Services are registered in bootstrap order (M1 first, M10 last). Shutdown hooks are stored in a list and popped (removing from end). So Intelligence (registered last) shuts down first, then Git, then Obsidian... down to the AI Manager. This is LIFO order. It makes sense because higher-level services depend on lower-level ones — you shut down the consumer before the producer.

---

11. Summary

What We Built

ProjectMind is a **10-module autonomous AI system** that acts as a developer's memory and documentation assistant. Every module has a single, clear responsibility and

communicates exclusively through an EventBus.

The 10 Modules at a Glance

Module	What It Does	Key Technology
M1 — Foundation	Config, logging, registry, bootstrap	Python stdlib
M2 — Watcher	Detects file changes in real time	watchdog
M3 — AI	Talks to Ollama/Qwen, manages prompts	ollama Python client
M4 — Analysis	Parses Python files into structured data	Python `ast` module
M5 — Docs	Generates rich markdown documentation	Jinja2
M6 — Graph	Builds dependency graph of the codebase	networkx
M7 — Memory	Stores and searches code knowledge	ChromaDB + sentence-transformers
M8 — Obsidian	Writes everything as linked markdown notes	atomic file I/O
M9 — Git	Monitors commits, creates AI summaries	subprocess + ChromaDB
M10 — Intelligence	Detects patterns, suggests refactors	AI + graph analysis

The Data Flow

```
File saved on disk
      ↓
M2 detects change (watchdog) → debounce → EventBus
      ↓
M4 analyzes the file (AST + AI) → FileAnalysis object → EventBus
      ↓
M5 generates documentation → markdown string → EventBus
M6 updates the dependency graph → graph.graph_updated
M7 embeds chunks into ChromaDB
      ↓
M8 writes the final note to Obsidian vault
      ↓
M10 (every 60 min) detects patterns → AI suggestions → SuggestionStore
```

Design Principles Applied

Principle	Where Applied
Single Responsibility	Each module does one thing
Open/Closed	New modules plug into EventBus without changing existing ones
Dependency Inversion	All modules depend on abstract interfaces, not concrete classes
Fail-Safe Defaults	Everything disabled by default in config
Defense in Depth	AI failures, file errors, service crashes all handled at multiple levels
Atomic Writes	No corrupt files on crash
LIFO Shutdown	Consumers stop before producers
Thread Safety	RLock on EventBus and Registry

Key Design Decision: Why All-Local?

ProjectMind runs entirely on your machine:

- No cloud API keys to manage
- No data leaves your system
- Works offline
- Free to use (Ollama models are open-weight)
- Latency is predictable (no network round trips)

The only external dependency is a running Ollama server on `localhost:11434`.

This documentation was written to teach ProjectMind from first principles. Every concept, class, function, and design decision has been explained in plain language. If you understand this document, you understand the entire codebase.